Backend - Laravel

Model

Rostagni Csaba

2026. november 9.

Tartalom

1. Eloquent
2. Query Builder vs Model
3. Lekérdezések
4. Strict mode

Tartalom

1. Eloquent
   - Bevezető
   - Modell generálása
   - Tábla nevének meghatározása
   - Kulcs meghatározása
   - Időbélyegek kezelése

Eloquent

- A Laravel beépített ORM megoldása
- Object Relation Mapping működése, Object Relation Mapper feladatai
  - "Probléma": Az RDBMS nem objektumokként tárolja az adatokat
  - A táblákhoz 1-1 model tartozik általában, de nem mindig
  - Az adatbázis egy rekordja egy objektum
  - Adatok betöltése az objektumba
  - Objektum módosításainak átvezetése az adatbázisba

Linkek:

- ORM - Wikipedia
- Laravel Dokumentáció: Eloquent

Tartalom

1. Eloquent
   - Bevezető
   - Modell generálása
   - Tábla nevének meghatározása
   - Kulcs meghatározása
   - Időbélyegek kezelése

Model generálása

CMD

```
php artisan make:model Book
```

PHP

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    //
}
```

Linkek:

- Laravel Dokumentáció: Model generálása

Model infók

- A `app/Models` mappában lesz alapértelmezetten
- A `Illuminate\Database\Eloquent\Model` osztályból származik
- `Book` modell - `books` tábla
- `BookAuthor` modell - `book_authors` tábla
- A modell egyesszámot és PascalCase-t használ
- A tábla többesszámot és snake_case-t használ
- A tábla neve az adott modellben felülbírálható, hogy az `Auto` modell adatai ne az `autos`, hanem az `autok` táblában legyen
- Alapértelmezetten a kulcsot `id`-nek és `int` típusúnak, továbbá automatikusan növekvőnek tekinti.

Tartalom

1. Eloquent
   - Bevezető
   - Modell generálása
   - Tábla nevének meghatározása
   - Kulcs meghatározása
   - Időbélyegek kezelése

Elsődleges kulcs

PHP
```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Attributes\Table;

#[Table(name:'macskak', key:'macska_id')]
class Cat extends Model {
    // ...
}
```

- A modellhez tartozó **tábla neve** a `Table` attribútum első (`name`) paraméterével módosítható
- A **tábla kulcsa** a `Table` attribútum második paramétervel (`key`) módosítható
- Összetett kulcsot a modell nem támogatja, kivéve (összekötő táblák)

Tartalom

1. Eloquent
   - Bevezető
   - Modell generálása
   - Tábla nevének meghatározása
   - Kulcs meghatározása
   - Időbélyegek kezelése

Tartalom

1. Eloquent
   - Bevezető
   - Modell generálása
   - Tábla nevének meghatározása
   - Kulcs meghatározása
   - Időbélyegek kezelése

Létrehozva, módosítva dátumok

PHP
```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Attributes\Table;

#[Table(timestamps: false)]
class Cat extends Model {
    // ...
}
```

A táblában alapértelmezetten egy `created_at` és egy `updated_at` oszlopokat vár el. Egy rekord beszúrásakor és módosításakor ezeket az értékeket szeretné módosítani, ez a modell feladata. Ha nincs ilyenünk, akkor ez hibáz okozhat!

- A `Table` attribútum 5. paramétere (`timestamps`) konfigurálja
- Alternatív megoldás lehet külön attribútum is: `Illuminate\Database\Eloquent\Attributes\WithoutTimestamps`

Létrehozva, módosítva dátumok

PHP
```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Cat extends Model {
    const CREATED_AT = 'letrehozas_datuma';
    const UPDATED_AT = 'modositas_datuma';
}
```

- A `created_at` és a `updated_at` oszlopokat átnevezhetjük

Tartalom

1. Eloquent
2. Query Builder vs Model
3. Lekérdezések
4. Strict mode

QueryBuilder vs Model

QueryBuilder

```php
DB::table('cars')
```

- A `DB::table()` egy `Illuminate\Database\Query\Builder`-t ad vissza
- A `table()` metódus paramétere határozza meg a tábla nevét

Model

```php
Car::query();
```

- A `Car` modell már tudja a tábla nevét (`$table` tulajdonság)
- A `Car::query()` egy `Illuminate\Database\Eloquent\Builder`-t ad vissza

Linkek:

- QueryBuilder
- Eloquent

QueryBuilder vs Model

QueryBuilder

```php
DB::table('cars')->get();
```

- A `get()` végrehajtja a lekérdezést és visszaadja az eredményt
  - `Illuminate\Support\Collection`-t ad vissza
  - A gyűjteményben `stdClass` típusú elemek lesznek

Model

```php
Car::query()->get();
```

- A `get()` végrehajtja a lekérdezést és visszaadja az eredményt
  - `Illuminate\Database\Eloquent\Collection`-t ad vissza
  - Az Eloquent gyűjteményben `Car` típusú elemek lesznek

QueryBuilder vs Model

QueryBuilder

```php
DB::table('cars')->first();
```

- A `first()` végrehajtja a lekérdezést és visszaadja az eredményt
  - `stdClass`-t ad vissza

Model

```php
Car::query()->first();
```

- A `first()` végrehajtja a lekérdezést és visszaadja az eredményt
  - `App\Models\Car`-t ad vissza

Tartalom

1. Eloquent
2. Query Builder vs Model
3. Lekérdezések
4. Strict mode

Tartalom

3 Lekérdezések
- Összes rekord lekérése
- Egy rekord rekérése
- Adat beszúrása
- Törlés

Egy tábla összes adatának lekérése modell segítségével

Laravel

```
use App\Models\Car;
$cars = Car::all();
dd($cars);
```

Eredmény

```
Illuminate\Database\Eloquent\Collection {#1025
    #items: array:4 [
        0 => App\Models\Car {#619 }
        1 => App\Models\Car {#987 }
        2 => App\Models\Car {#1234 }
        3 => App\Models\Car {#1235 }
    ]
    #escapeWhenCastingToString: false
}
```

Collection osztály (Eloquent)

- Több elem visszaadásakor `Illuminate\Database\Eloquent\Collection` osztály egy példányát kapjuk
- Az osztály a `Illuminate\Support\Collection` osztály leszármazottja
- Az ősosztály metódusait is lehet alkalmazni
- Az ősosztály metódsait kibővíti
- Amit lehet az adatbázis végezzen el, ne utólag a Collection valamelyik metódusa!
- A legtöbb metódus az Eloquent Collection osztályát adja vissza, kivéve ami "sima" Collection osztályt ad vissza (pl.: `pluck()`, `toBase()`, `modelKeys()`)

Linkek:

- Laravel Dokumentáció: Eloquent Collections
- Laravel Dokumentáció: Eloquent Collections kibővített metódusok

A QueryBuilder használata Eloquent modelleken

Feladat: Jelenítse meg az Opel gyártmányú járműveket

Query Builder

```
$opelek = DB::table("cars")
            ->where('manufacturer','Opel')
            ->get();
dd($opelek);
```

Eloquent

```
$opelek = Car::where('manufacturer','Opel')->get();
dd($opelek);
```

- A modell tudja, hogy mi a tábla neve
- A `query()` a legtöbb esetben elhagyható

Query Builder eredmény

Eredmény

```
Illuminate\Support\Collection {#311
#items: array:1 [
    0 => {#313
    +"plate_number": "XXX-111"
    +"manufacturer": "Opel"
    +"model": "Adam"
    +"category": "M1"
    +"fuel": "benzin"
    }
]
#escapeWhenCastingToString: false
}
```

Eloquent eredmény

Eredmény

```
Illuminate\Database\Eloquent\Collection {#1100
    #items: array:1 [
        0 => App\Models\Car {#1032
        #primaryKey: "plate_number"
        #keyType: "string"
        +incrementing: false
        #connection: "mysql"
        #table: "cars"
        ...
        #attributes: array:5 [>]
        ...
        }
    ]
    #escapeWhenCastingToString: false
}
```

- Az `attributes` tartalmazza egy rekord jelenlegi adatait

Tartalom

3 Lekérdezések
- Összes rekord lekérése
- Egy rekord rekérése
- Adat beszúrása
- Törlés

Egy model betöltése ID alapján

```php
$product = Product::where('id',1)->first();
```

- A `where()` segítségével szűrhetünk az ID-re
  - Az alapértelmezett művelet az egyenlőség, így a műveleti jel elhagyható
- A `first()` az első találatot adja vissza

```php
$product = Product::firstWhere('id',1);
```

- A `firstWhere()` lerövidíti a `where()` és `first()` használatát
- A `firstWhere()` kimenete a **megadott osztály példánya** lesz, NEM `Collection`

A find() metódus

```php
$product = Product::find(1)
```

- A `find()` az elsődleges kulcsot várja paraméternek
- Mivel csak egy rekord létezhet a megadott kulccsal, így egy elemet ad vissza
- A kiválasztott modell (`Product`) egy példányát adja vissza feltöltve a megfelelő rekord adataival
- Amennyiben az adott kulcs nem található, úgy `null` értéket ad vissza

A find() metódus

```php
$car = Book::find('9781398704411')
```

- A QueryBuilder esetében csak `id` alapján tud szűrni a `find`
- A **modell** esetében meg lehet adni az elsődleges kulcs nevét, így azt használja
- Így nem az `id`, hanem a `isbn` mezőre keres rá

A firstOrFail() metódus

```php
$product = Product::where('id',1)->firstOrFail();
```

- A kiválasztott modell egy példányát adja vissza feltöltve a megfelelő rekord adataival
- Amennyiben az adott feltételnek megfelelő rekord nem található, úgy kivételt dob:
  `Illuminate\Database\Eloquent\ModelNotFoundException`
- A Controllerben kezeli a kivételt és 404-es hibaoldal lesz ez eredmény

Linkek:

- Laravel API: Eloquent - firstOrFail()

A findOrFail() metódus

```php
$car = Car::findOrFail("ABC-123")
```

- A `findOrFail()` az elsődleges kulcsot várja paraméternek
- Mivel csak egy rekord létezhet a megadott kulccsal, így egy elemet ad vissza
- A kiválasztott modell egy példányát adja vissza feltöltve a megfelelő rekord adataival
- Amennyiben az adott kulcs nem található, úgy kivételt dob: `Illuminate\Database\Eloquent\ModelNotFoundException`
- A Controllerben kezeli a kivételt és 404-es hibaoldal lesz ez eredmény

Linkek:

- Laravel API: Eloquent - findOrFail()

Tartalom

3 Lekérdezések
- Összes rekord lekérése
- Egy rekord rekérése
- Adat beszúrása
- Törlés

Új rekord létrehozása - save()

PHP

```php
$car = new Car();
$car->owner = 1;
$car->manufacturer_id = 1;
$car->model = "Fiesta";
$car->plate_number = "ASD-999";
$car->category = "M1";
$car->fuel = "benzin";
$car->consumption = 7.9;
$car->color = "piros";

$car->save();
```

- A `save()` elmenti az objektumot az adatbázisba
- A mentés után megkapja az aktuális `id` értéket

Új rekord létrehozása - create()

PHP

```php
$data = [
    'plate_number' => 'DDD-111',
    'manufacturer_id' => 1,
    'model' => 'Ka',
    'category' => 'M1',
    'fuel' => 'benzin',
    'color' => 'fehér',
    'consumption' => 5.9,
    'owner' => 1,
];

$car = Car::create($data);
```

- A `create()` metódus egy rekord több értékét szúrja be
- Ami nem szerepel a modell `protected $fillable` tulajdonságában, azt nem menti el a `create`

Mass Assignment

PHP
```php
class Car extends Model
{
    ...
    protected $fillable = [
        'plate_number', 'manufacturer_id', 'model', 'category',
        'fuel', 'color','consumption','owner'
    ];
    ...
}
```

- A felhasználó egy HTTP kérésben olyan adatot is küldhet, aminek nem kellene az adatbázisba bekerülnie
  - például `admin=1` és már admin jogosultságot kap egy regisztrációval
- A modell alapértelmezetten nem fogad el tetszőleges adatot
- A `$fillable` tulajdonság határozza meg, hogy mit lehet (white list) a `$guarded`, hogy mit tilos (black list) tömegesen beállítani. Nem keverhető!

Linkek:

Tartalom

3 Lekérdezések
- Összes rekord lekérése
- Egy rekord rekérése
- Adat beszúrása
- Törlés

Törlés

```php
$cat= Cat::findOrFail(3);
$cat->delete();
```

- Keressük meg a törlendő modellt
- A `delete()` metódus törli a rekordot

SoftDeletes

```php
use Illuminate\Database\Eloquent\SoftDeletes;

class Car extends Model
{
    use SoftDeletes;
    ...
}
```

- A `SoftDeletes` trait kibővíti többek között `withTrashed` metódussal
- A modell törlésekor csak a `deleted_at` értékét tölti ki
- A továbbiakban alapértelmezetten a "törölteket" NEM jeleníti meg, ha modellen keresztül próbáljuk meg elérni

```php
Car::all(); // Töröltek nélkül
Car::withTrashed()->get(); // Töröltekkel együtt
```

Tartalom

1. Eloquent
2. Query Builder vs Model
3. Lekérdezések
4. Strict mode

Model::shouldBeStrict()

PHP

```php
use Illuminate\Database\Eloquent\Model;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Model::shouldBeStrict();
    }
}
```

- Az `app/Providers/AppServiceProvider.php` fájlba kell elhelyezni a `Model::shouldBeStrict()`-et.
- Három dolgot tesz
  - Megakadályozza a LazyLoading-ot `Model::preventLazyLoading();`
  - Mentéskor jelzi, ha valamelyik adatot eldobná mentés helyett `Model::preventSilentlyDiscardingAttributes();`
  - Jelzi, ha olyan mezőt szeretnénk elérni/kiírni, ami nem létezik / nem lett lekérve `Model::preventAccessingMissingAttributes();`
- A generált hibaüzenetek csökkentik a rejtett hibák előfordulásait

Linkek:

- shouldBeStrict - Laravel News